

Policy Gradient

Policy gradient refers to a family of algorithms that use

- a parameterized class of functions to represent the policy, and
- gradient-based optimization techniques to optimize the policy.

Let $\Pi = \{\pi_\theta : \theta \in \mathbb{R}^d\}$ represent a parameterized class of policies.

Searching for an optimal policy within this class can be formulated as

$$\begin{array}{ccc} \pi^* & = \arg \max_{\pi \in \Pi} & \underbrace{E_{\mathcal{M}_0, P, \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, A_t) \right]}_{V^\pi(\mathcal{M}_0) = E_{s_0 \sim \mathcal{M}_0} [V^\pi(s_0)]} \\ \downarrow & \downarrow & \\ \pi_{\theta^*} & \theta \in \Theta & \end{array}$$

or equivalently as

$$\begin{array}{ccc} \theta^* & = \arg \max_{\theta \in \Theta} & J(\pi_\theta) \\ \downarrow & \theta \in \Theta \rightarrow \text{e.g., } \mathbb{R}^d & \\ \pi^* = \pi_{\theta^*} & \pi \in \Pi & \end{array} ,$$

where

$$J(\pi_\theta) = V^{\pi_\theta}(\mathcal{M}_0) .$$

Example parameterized classes of functions for policy representation:

Softmax policy: for (finite) discrete state and action spaces

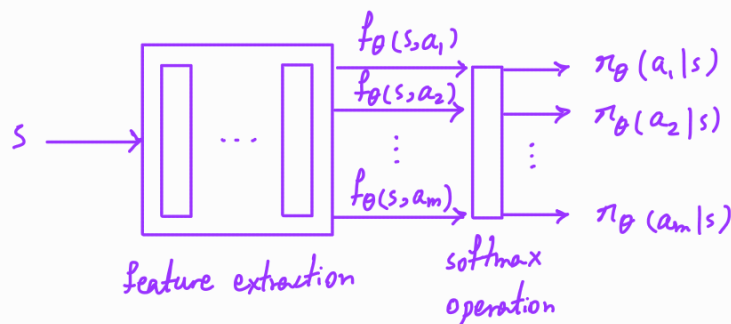
$$\pi_{\theta}(a|s) = \frac{\exp(\theta_{s,a})}{\sum_{a' \in A} \exp(\theta_{s,a'})} \quad \theta_{s,a} \in \mathbb{R} \quad \forall s \in S, \forall a \in A$$

Softmax linear policy: for large discrete or continuous state and action spaces

$$\pi_{\theta}(a|s) = \frac{\exp(\theta^T \phi(s,a))}{\sum_{a' \in A} \exp(\theta^T \phi(s,a'))} \quad \theta \in \mathbb{R}^K, \quad \phi(s,a) \in \mathbb{R}^K \quad \forall s \in S, \forall a \in A$$

Softmax neural policy: for large discrete or continuous state and action spaces

$$\pi_{\theta}(a|s) = \frac{\exp(f_{\theta}(s,a))}{\sum_{a' \in A} \exp(f_{\theta}(s,a'))} \quad \theta \in \mathbb{R}^K, \quad f_{\theta}: S \times A \rightarrow \mathbb{R}$$



Stochastic gradient descent (ascent) for policy optimization

considering the objective function

$$J(\pi_\theta) = V^{\pi_\theta}(\mu_0) = \mathbb{E}_{\mu_0, P, \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \right],$$

we want to apply gradient-based maximization, i.e.,

$$\theta_{t+1} = \theta_t + \eta \underbrace{\nabla_{\theta} J(\pi_{\theta_t})}_{\nabla_{\theta} J(\pi_{\theta}) \big|_{\theta=\theta_t}}.$$

parameter update

However, the gradient of this function is not computable analytically since

- the model is not known, and
- there is an infinite summation in the objective.

Even, this function itself is not computable analytically. However, we can estimate it through sampling-based techniques.

Idea: One can use sampling-based techniques to also estimate the gradient and use SGD for updating the parameters.

REINFORCE :

Let $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ be a sample trajectory with probability $P[\tau | \mu_0, \pi, P] \leftarrow$

$$p_{\theta}(\tau) = \mu_0(s_0) \prod_{t=0}^{\infty} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

Then, we have

$$J(\pi_{\theta}) = \underbrace{\mathbb{E}_{\mu_0, \pi_{\theta}, P}}_{\mathbb{E}_{\tau \sim p_{\theta}(\tau)}} \left[\underbrace{\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)}_{R(\tau)} \right] .$$

Taking the gradient, we have

$$\nabla_{\theta} J(\pi_{\theta}) = \nabla_{\theta} \left[\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)] \right] .$$

Question : How can we use sampling techniques to estimate $\nabla_{\theta} J(\pi_{\theta})$?

Previously, we learned how to apply Monte Carlo methods to estimate $\mathbb{E}_{x \sim p} [f(x)]$ by

$$\hat{\mathbb{E}}_{x \sim p} [f(x)] = \frac{1}{n} \sum_{i=1}^n f(x_i) .$$

However, here we need to estimate $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}} [f(x)]$.

$$\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}} [f(x)] = \nabla_{\theta} \left[\sum_{x \in \mathcal{X}} p_{\theta}(x) f(x) \right] = \sum_{x \in \mathcal{X}} \nabla_{\theta} (p_{\theta}(x) f(x)) = \sum_{x \in \mathcal{X}} \nabla_{\theta} p_{\theta}(x) f(x)$$

* The log-derivative trick enables us to apply Monte Carlo methods to estimate $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}} [f(x)]$.

Recall that

$$\nabla_{\theta} J(\pi_{\theta}) = \nabla_{\theta} \left[\mathbb{E}_{z \sim p_{\theta}(z)} \left[\underbrace{\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)}_{R(z)} \right] \right]$$

definition of expectation $\leftarrow = \nabla_{\theta} \left[\sum_{z \in \mathcal{Z}} p_{\theta}(z) R(z) \right]$

linearity of $\leftarrow$ $= \sum_{z \in \mathcal{Z}} \nabla_{\theta} [p_{\theta}(z) R(z)] = \sum_{z \in \mathcal{Z}} \nabla_{\theta} p_{\theta}(z) R(z)$
gradient

multiply and $\leftarrow = \sum_{z \in \mathcal{Z}} \frac{\nabla_{\theta} p_{\theta}(z)}{p_{\theta}(z)} p_{\theta}(z) R(z)$
div. out

derivative of $\leftarrow = \sum_{z \in \mathcal{Z}} \nabla_{\theta} \log(p_{\theta}(z)) p_{\theta}(z) R(z)$
 $\log(\cdot)$

definition of $\leftarrow = \mathbb{E}_{z \sim p_{\theta}(z)} [\nabla_{\theta} \log(p_{\theta}(z)) R(z)]$
expectation

definition of $\leftarrow = \mathbb{E}_{z \sim p_{\theta}(z)} \left[\nabla_{\theta} \left[\log \mathcal{M}_0(s_0) + \sum_{t=0}^{\infty} \log \pi_{\theta}(a_t | s_t) \right. \right.$
 $p_{\theta}(z)$ $\left. \left. + \sum_{t=0}^{\infty} \log P(s_{t+1} | s_t, a_t) \right] R(z) \right]$

simplifying $\leftarrow = \mathbb{E}_{z \sim p_{\theta}(z)} \left[\left(\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) R(z) \right]$

So far, we have seen that for the objective function

$$J(\pi_\theta) = \mathbb{E}_{z \sim p_\theta(z)} \left[\underbrace{\sum_{t=0}^{\infty} \gamma^t R(s_t, A_t)}_{R(z)} \right],$$

the gradient can be expressed as

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{z \sim p_\theta(z)} \left[\left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) \right) R(z) \right].$$

Note: In this form, the gradient

- does not need the derivative of an infinite product, and
- does not explicitly depend on the (unknown) model.

Suppose we obtain a sample trajectory z_i drawn from $p_\theta(z)$,

i.e., according to $\mathcal{M}$, P , and π_θ . Then, we can compute

$$g_i = \left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) R(z_i).$$

as an estimate of the gradient that is unbiased:

$$\mathbb{E}_{z_i \sim p_\theta(z)} [g_i] = \nabla_\theta J(\pi_\theta).$$

REINFORCE algorithm

- Initialize θ_0
- For $i = 0, 1, 2, \dots$
 - Sample trajectory z_i according to μ_0 , P , and π_{θ_i}
 - Estimate gradient by

$$g_i = \left(\sum_{t=0}^{\infty} \nabla_{\theta} \log \pi_{\theta_i}(a_t^i | s_t^i) \right) R(z_i)$$

- Update θ

$$\theta_{i+1} = \theta_i + \eta g_i$$

Variance Reduction

Considering the objective function

$$J(\pi_\theta) = \mathbb{E}_{z \sim p_\theta(z)} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, A_t) \right],$$

the gradient estimate

$$g_i = \left(\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) R(z_i)$$

based on a sample trajectory z_i , while unbiased, can lead to high variance

$$\mathbb{E}_{z_i \sim p_\theta(z)} [g_i] = \nabla_\theta J(\pi_\theta), \text{ and}$$

$$\mathbb{E}_{z_i \sim p_\theta(z)} [\|g_i - \nabla_\theta J(\pi_\theta)\|_2^2] \text{ high}.$$

* One can use variance reduction techniques to tackle this issue.

• using the state-action value function



actor-critic
method

• subtracting a baseline from the gradient

advantage actor-critic
method

Actor - Critic: Recall that the objective function is

$$\begin{aligned} J(\pi_\theta) &= V^{\pi_\theta}(\mu_0) \\ &= \mathbb{E}_{\substack{s_0 \sim \mu_0 \\ a_0 \sim \pi_\theta(\cdot|s_0)}} [Q^{\pi_\theta}(s_0, a_0)] . \end{aligned}$$

Taking the gradient, we have

$$\begin{aligned} \nabla_\theta J(\pi_\theta) &= \nabla_\theta \mathbb{E}_{s_0 \sim \mu_0} [V^{\pi_\theta}(s_0)] \\ &= \nabla_\theta \mathbb{E}_{\substack{s_0 \sim \mu_0 \\ a_0 \sim \pi_\theta(\cdot|s_0)}} [Q^{\pi_\theta}(s_0, a_0)] \\ &= \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_\theta \mathbb{E}_{a_0 \sim \pi_\theta(\cdot|s_0)} [Q^{\pi_\theta}(s_0, a_0)] \right] \end{aligned}$$

definition of $\leftarrow$ expectation $= \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_\theta \left[\sum_{a_0 \in \mathcal{A}} \pi_\theta(a_0|s_0) Q^{\pi_\theta}(s_0, a_0) \right] \right]$

$$\begin{aligned} &= \mathbb{E}_{s_0 \sim \mu_0} \left[\sum_{a_0 \in \mathcal{A}} (\nabla_\theta \pi_\theta(a_0|s_0) Q^{\pi_\theta}(s_0, a_0) \right. \\ &\quad \left. + \pi_\theta(a_0|s_0) \nabla_\theta \underline{Q^{\pi_\theta}(s_0, a_0)}) \right] \end{aligned}$$

log-derivative $\leftarrow$ trick $= \mathbb{E}_{s_0 \sim \mu_0} \left[\sum_{a_0 \in \mathcal{A}} \pi_\theta(a_0|s_0) \nabla_\theta \log \pi_\theta(a_0|s_0) Q^{\pi_\theta}(s_0, a_0) \right.$

+ Bellman consistency equation $\left. + \sum_{a_0 \in \mathcal{A}} \pi_\theta(a_0|s_0) \nabla_\theta \left[\underline{R(s_0, a_0) + \gamma \mathbb{E}_{s_1 \sim \mathcal{P}(\cdot|s_0, a_0)} [V^{\pi_\theta}(s_1)]} \right] \right]$

$$= \mathbb{E}_{s_0 \sim \mu_0} \left[\sum_{a_0 \in A} \pi_{\theta}(a_0 | s_0) \nabla_{\theta} \log \pi_{\theta}(a_0 | s_0) Q^{\pi_{\theta}}(s_0, a_0) + \gamma \sum_{a_0 \in A} \pi_{\theta}(a_0 | s_0) \mathbb{E}_{s_1 \sim P(\cdot | s_0, a_0)} [\nabla_{\theta} V^{\pi_{\theta}}(s_1)] \right]$$

definition of $\leftarrow$ expectation $= \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_{\theta} \log \pi_{\theta}(a_0 | s_0) Q^{\pi_{\theta}}(s_0, a_0) \right]$
 $a_0 \sim \pi_{\theta}(\cdot | s_0)$

$$+ \gamma \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_{\theta} V^{\pi_{\theta}}(s_1) \right]$$

$a_0 \sim \pi_{\theta}(\cdot | s_0)$
 $s_1 \sim P(\cdot | s_0, a_0)$

repeat the argument $\leftarrow = \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_{\theta} \log \pi_{\theta}(a_0 | s_0) Q^{\pi_{\theta}}(s_0, a_0) \right]$
 $a_0 \sim \pi_{\theta}(\cdot | s_0)$

$$+ \gamma \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_{\theta} \log \pi_{\theta}(a_1 | s_1) Q^{\pi_{\theta}}(s_1, a_1) \right]$$

$a_0 \sim \pi_{\theta}(\cdot | s_0)$
 $s_1 \sim P(\cdot | s_0, a_0)$
 $a_1 \sim \pi_{\theta}(\cdot | s_1)$

$$+ \gamma^2 \mathbb{E}_{s_0 \sim \mu_0} \left[\nabla_{\theta} V^{\pi_{\theta}}(s_2) \right]$$

$a_0 \sim \pi_{\theta}(\cdot | s_0)$
 $s_1 \sim P(\cdot | s_0, a_0)$
 $a_1 \sim \pi_{\theta}(\cdot | s_1)$
 $s_2 \sim P(\cdot | s_1, a_1)$

$=$
 $\vdots$

after infinite iterations $\leftarrow = \sum_{t=0}^{\infty} \gamma^t \mathbb{E}_{z \sim p_{\theta}(z)} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{Q^{\pi_{\theta}}(s_t, a_t)}_{f(s_t, A_t)} \right]$

$$= \mathbb{E}_{z \sim p_{\theta}(z)} \left[\sum_{t=0}^{\infty} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) Q^{\pi_{\theta}}(s_t, a_t) \right]$$

$$= \frac{1}{1-\gamma} \mathbb{E}_{s, a \sim \bar{\pi}_{\mu_0}^{\pi}} [f(s, a)] \leftarrow = \frac{1}{1-\gamma} \mathbb{E}_{s, a \sim \bar{\pi}_{\mu_0}^{\pi}} \left[\nabla_{\theta} \log \pi_{\theta}(a | s) Q^{\pi_{\theta}}(s, a) \right]$$

Then, we can form a gradient estimate as

$$g_i = \frac{1}{1-\gamma} \nabla_{\theta} \log \pi_{\theta}(a^i | s^i) Q^{\pi_{\theta}}(s^i, a^i),$$

which is unbiased

$$\mathbb{E}_{s^i, a^i \sim \bar{\pi}_{\mu_0}^{\pi_{\theta}}} [g_i] = \nabla_{\theta} J(\pi_{\theta}).$$

Actor-critic algorithm

- Initialize θ_0
- For $i = 0, 1, 2, \dots$

actor π_{θ}

critic $Q^{\pi_{\theta}} \approx Q_{\phi}$

→ learned with supervised learning

- Sample trajectory according to μ_0 , P , and π_{θ_i} to arrive at (s^i, a^i)
- Estimate gradient by

$$g_i = \frac{1}{1-\gamma} \nabla_{\theta} \log \pi_{\theta_i}(a^i | s^i) Q^{\pi_{\theta_i}}(s^i, a^i)$$

- Update θ

$$\theta_{i+1} = \theta_i + \eta g_i$$

Advantage Actor-Critic: Recall that the gradient of the objective function can be written as

$$\nabla_{\theta} J(\pi_{\theta}) = \frac{1}{1-\gamma} \mathbb{E}_{s,a \sim \bar{\tau}_{\mu_0}^{\pi_{\theta}}} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi_{\theta}}(s,a) \right].$$

Now consider an arbitrary state-based function $b: S \rightarrow \mathbb{R}$.

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot|s)} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) b(s) \right]$$

definition of $\leftarrow$ expectation $= \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) b(s)$

derivative of $\leftarrow$ $\log(\cdot)$ $= \sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \frac{\nabla_{\theta} \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)} b(s)$

$$= \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s) b(s) = b(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a|s)$$

linearity of $\leftarrow$ gradient $= b(s) \nabla_{\theta} \left[\sum_{a \in \mathcal{A}} \pi_{\theta}(a|s) \right] = b(s) \nabla_{\theta} [1] = 0$

Therefore, we can subtract $b(s)$ from $Q^{\pi_{\theta}}(s,a)$ in the gradient expression:

$$\nabla_{\theta} J(\pi_{\theta}) = \frac{1}{1-\gamma} \mathbb{E}_{s,a \sim \bar{\tau}_{\mu_0}^{\pi_{\theta}}} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) (Q^{\pi_{\theta}}(s,a) - b(s)) \right].$$

This result holds for any arbitrary function $b(s)$. In practice, the common choice is the value function $v_{(s)}^{\pi_{\theta}}$.

$$\nabla_{\theta} J(\pi_{\theta}) = \frac{1}{1-\gamma} \mathbb{E}_{s,a \sim \bar{\tau}_{\mu_0}^{\pi_{\theta}}} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) \underbrace{Q^{\pi_{\theta}}(s,a) - v^{\pi_{\theta}}(s)}_{\text{Advantage}} \right].$$

Then, we can form a gradient estimate as

$$g_i = \frac{1}{1-\gamma} \nabla_{\theta} \log \pi_{\theta}(a^i | s^i) A^{\pi_{\theta}}(s^i, a^i),$$

which is unbiased

$$\mathbb{E}_{s^i, a^i \sim \bar{\pi}_{\theta_0}^{\pi_{\theta}}}[g_i] = \nabla_{\theta} J(\pi_{\theta}).$$

Advantage Actor-critic algorithm

- Initialize θ_0
- For $i = 0, 1, 2, \dots$

actor π_{θ}

critic $Q^{\pi_{\theta}} \approx Q_{\phi}$

→ learned with supervised learning

- Sample trajectory according to μ_0 , P , and π_{θ_i} to arrive at (s^i, a^i)
- Estimate gradient by

$$g_i = \frac{1}{1-\gamma} \nabla_{\theta} \log \pi_{\theta_i}(a^i | s^i) A^{\pi_{\theta_i}}(s^i, a^i)$$

- Update θ

$$\theta_{i+1} = \theta_i + \eta g_i$$

Note: Subtracting this baseline from the gradient estimate reduces its variance, improving the convergence of the stochastic gradient ascent algorithm.

REINFORCE

π_θ

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{z \sim p_\theta(z)} \left[\left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t | s_t) \right) R(z) \right]$$

$$g_i = \left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right) R(z_i)$$

Actor-Critic $\rightarrow AC$

π_θ, Q_ϕ

$$\nabla_\theta J(\pi_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s, a \sim \bar{r}_{\pi_\theta}^{\pi_\theta}} \left[\nabla_\theta \log \pi_\theta(a_t | s_t) Q^{\pi_\theta}(s_t, a_t) \right]$$

$$g_i = \frac{1}{1-\gamma} \nabla_\theta \log \pi_{\theta_i}(a^i | s^i) Q_{\phi}^{\pi_{\theta_i}}(s^i, a^i)$$

Advantage Actor-Critic $\rightarrow A2C$

π_θ, Q_ϕ

$$\nabla_\theta J(\pi_\theta) = \frac{1}{1-\gamma} \mathbb{E}_{s, a \sim \bar{r}_{\pi_\theta}^{\pi_\theta}} \left[\nabla_\theta \log \pi_\theta(a_t | s_t) A^{\pi_\theta}(s_t, a_t) \right]$$

$$g_i = \frac{1}{1-\gamma} \nabla_\theta \log \pi_{\theta_i}(a^i | s^i) A_{\phi}^{\pi_{\theta_i}}(s^i, a^i)$$

Note [Variants of policy gradient algorithms]: These policy gradient algorithms have several variants that aim to improve performance, accelerate the learning process, or extend the setting, such as

- Soft Actor-Critic (SAC)

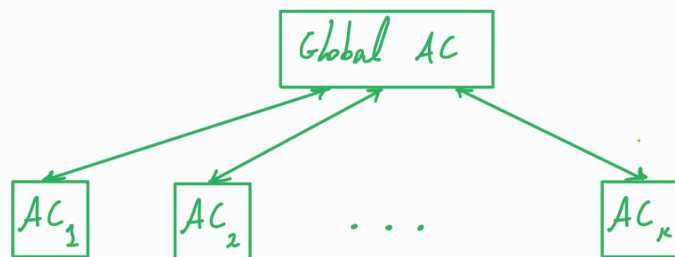
Augmented objective based on the maximum entropy principle

$$\begin{aligned} X &\sim P_X \\ \mathcal{H}(X) &= \mathcal{H}(P_X) \quad \text{entropy} \\ &= \mathbb{E}_{X \sim P_X} [-\log(P_X(X))] \\ &\quad \text{average uncertainty} \end{aligned}$$

$$J(\pi_\theta) = \mathbb{E}_{M_0, P, \pi_\theta} \left[\underbrace{\sum_{t=0}^{\infty} \gamma^t (R(s_t, A_t))}_{\text{AC objective}} + \underbrace{\lambda \mathcal{H}(\pi_\theta(\cdot | s_t))}_{\text{entropy of policy}} \right]$$

- Asynchronous Advantage Actor-Critic (A3C)

parallelized learning process



- Deep Deterministic policy Gradient (DDPG)

capable of dealing with continuous action space

by making Q_ϕ differentiable with respect to policy